

目录

阶段 05 系统调用实验报告	2
一、实验基本信息	2
(一) 实验目的	2
(二) 实验环境	2
二、实验核心原理	2
(一) 系统调用完整流程	2
(二) RISC-V 系统调用关键机制	2
三、实验步骤与关键代码	3
(一) 任务 1: 系统调用框架设计	3
(二) 任务 2: 参数提取与安全检查	4
(三) 任务 3: 基础系统调用实现	5
(四) 任务 4: 用户态系统调用接口实现	6
(五) 任务 5: 系统调用安全性增强	7
(六) 任务 6: 测试与调试实现	8
四、实验关键问题解答	9
(一) 系统调用核心机制问题	9
(二) 系统调用分发与参数传递问题	9
(三) 性能与安全问题	10
(四) 设计权衡与扩展性问题	10
(五) 错误处理问题	10

阶段 05 系统调用实验报告

[仓库地址在此处](#)

此阶段代码请通过 `git checkout syscall_simple` 来查看。

一、实验基本信息

(一) 实验目的

1. 深入分析 xv6 操作系统的系统调用机制，理解用户态与内核态的交互原理。
2. 设计并实现完整的系统调用框架，包括分发机制、参数传递及错误处理。
3. 完成进程控制、文件操作、内存管理类基础系统调用的开发，保障系统调用安全性与稳定性。
4. 解答实验指导中的核心问题，深化对系统调用性能、安全及扩展性的理解。

(二) 实验环境

- 操作系统：xv6
- 架构：RISC-V
- 开发工具：GCC、QEMU（模拟器）、Perl（桩代码生成）、GDB（调试）

二、实验核心原理

(一) 系统调用完整流程

1. 用户程序调用：用户通过库函数（如 `write()`）发起系统调用。
2. 桩代码执行：`usys.S` 中的桩代码将系统调用号加载到 `a7` 寄存器，执行 `ecall` 指令陷入内核。
3. 特权级切换：硬件自动完成 U 态到 S 态转换，保存 `pc` 到 `sepc`、设置 `scause` 为系统调用编码，跳转至内核陷阱处理入口。
4. 陷阱处理：内核执行 `uservec`→`usertrap` 流程，识别系统调用类型后调用 `syscall()` 分发函数。
5. 系统调用实现：根据系统调用号调用对应内核实现函数（如 `sys_write()`）。
6. 返回用户态：执行 `usertrapret`→`userret` 流程，恢复用户态寄存器和页表，通过 `sret` 指令返回用户程序。

(二) RISC-V 系统调用关键机制

- `ecall` 指令：触发特权级切换，从用户态进入监督态，触发内核陷阱处理。
- `scause` 寄存器：存储陷阱原因，系统调用对应编码为 8。
- `sepc` 寄存器：保存触发 `ecall` 时的用户程序计数器（`pc`），用于返回用户态时恢复执行位置。
- 寄存器传递：参数通过 `a0-a5` 寄存器传递，系统调用号通过 `a7` 寄存器传递，返回值通过 `a0` 寄存器返回。

三、实验步骤与关键代码

(一) 任务 1：系统调用框架设计

1. 系统调用描述符与系统调用表

```

1  // 系统调用描述符结构
2  struct syscall_desc {
3      int (*func)(void);      // 系统调用实现函数指针
4      char *name;             // 系统调用名称
5      int arg_count;          // 参数个数
6  };
7
8  // 系统调用编号定义
9  #define SYS_fork    1
10 #define SYS_exit    2
11 #define SYS_wait    3
12 #define SYS_kill    4
13 #define SYS_getpid  5
14 #define SYS_open    6
15 #define SYS_close   7
16 #define SYS_read    8
17 #define SYS_write   9
18 #define SYS_sbrk    10
19
20 // 系统调用实现函数声明
21 int sys_fork(void);
22 int sys_exit(void);
23 int sys_wait(void);
24 int sys_kill(void);
25 int sys_getpid(void);
26 int sys_open(void);
27 int sys_close(void);
28 int sys_read(void);
29 int sys_write(void);
30 void* sys_sbrk(void);
31
32 // 系统调用表
33 struct syscall_desc syscall_table[] = {
34     {NULL, "invalid", 0},          // 0号为无效调用
35     {sys_fork, "fork", 0},         // SYS_fork
36     {sys_exit, "exit", 1},        // SYS_exit
37     {sys_wait, "wait", 1},        // SYS_wait
38     {sys_kill, "kill", 1},        // SYS_kill
39     {sys_getpid, "getpid", 0},     // SYS_getpid
40     {sys_open, "open", 2},        // SYS_open
41     {sys_close, "close", 1},      // SYS_close
42     {sys_read, "read", 3},        // SYS_read
43     {sys_write, "write", 3},      // SYS_write

```

```

44     {sys_sbrk, "sbrk", 1}                // SYS_sbrk
45 };
46
47 #define NSYSCALLS (sizeof(syscall_table)/sizeof(syscall_table[0]))

```

2. 系统调用分发器实现

```

1  void syscall_dispatch(void) {
2      struct proc *p = myproc();
3      int num = p->trapframe->a7;  // 从a7寄存器获取系统调用号
4      int ret = -1;
5
6      // 验证系统调用号有效性
7      if (num > 0 && num < NSYSCALLS && syscall_table[num].func != NULL) {
8          // 调用对应系统调用实现函数
9          ret = syscall_table[num].func();
10     } else {
11         // 处理无效系统调用
12         printf("PID %d: invalid syscall %d\n", p->pid, num);
13     }
14
15     // 保存返回值到a0寄存器
16     p->trapframe->a0 = ret;
17 }

```

(二) 任务 2: 参数提取与安全检查

1. 参数提取辅助函数

```

1  // 从寄存器中提取整数参数 (n为参数索引, 0对应a0, 1对应a1...)
2  int get_syscall_arg(int n, long *arg) {
3      struct proc *p = myproc();
4      uint64 *regs = (uint64*)&p->trapframe->a0;  // a0-a5寄存器存储参数
5
6      if (n < 0 || n >= 6) {  // RISC-V最多支持6个寄存器参数
7          return -1;
8      }
9
10     *arg = regs[n];
11     return 0;
12 }
13
14 // 读取用户态字符串到内核缓冲区, 防止越界
15 int get_user_string(const char __user *str, char *buf, int max) {
16     struct proc *p = myproc();
17     if (max <= 0) return -1;
18
19     // 验证用户指针有效性并复制数据
20     if (copyin(p->pagetable, buf, (uint64)str, max) < 0) {

```

```
21     return -1;
22 }
23
24 // 确保字符串以'\0'终止
25 buf[max-1] = '\0';
26 return 0;
27 }
28
29 // 检查用户指针有效性（地址范围、权限）
30 int check_user_ptr(const void *ptr, int size) {
31     struct proc *p = myproc();
32     uint64 addr = (uint64)ptr;
33
34     // 检查地址是否在用户空间（xv6用户空间地址上限为0x80000000）
35     if (addr + size < addr || addr + size > 0x80000000) {
36         return -1;
37     }
38
39     // 检查内存页权限（是否可读/可写）
40     return walkaddr(p->pagetable, addr) == 0 ? -1 : 0;
41 }
```

（三）任务 3：基础系统调用实现

1. 进程控制类：sys_getpid

```
1 int sys_getpid(void) {
2     return myproc()->pid; // 直接返回当前进程PID
3 }
```

2. 文件操作类：sys_write

```
1 int sys_write(void) {
2     long fd, count;
3     char *buf;
4     struct proc *p = myproc();
5
6     // 提取参数（fd：文件描述符，buf：数据缓冲区，count：写入长度）
7     if (get_syscall_arg(0, &fd) < 0 || get_syscall_arg(1, (long*)&buf) <
8         0 || get_syscall_arg(2, &count) < 0) {
9         return -1;
10    }
11
12    // 参数有效性检查
13    if (fd < 0 || fd >= NOFILE || count < 0) {
14        return -1;
15    }
16    if (check_user_ptr(buf, count) < 0) {
17        return -1; // 无效用户指针
18    }
```

```

17     }
18
19     // 调用内核文件写入函数实现功能
20     return fwrite(p->ofile[fd], buf, count);
21 }

```

3. 内存管理类: sys_sbrk

```

1  void* sys_sbrk(void) {
2      long n;
3      struct proc *p = myproc();
4      uint64 old_heap_top = p->heap_top;
5
6      // 提取参数 (n为堆大小调整值, 正数扩展, 负数收缩)
7      if (get_syscall_arg(0, &n) < 0) {
8          return (void*)-1;
9      }
10
11     // 检查堆扩展范围合法性
12     if (p->heap_top + n > p->sz) {
13         // 分配新的物理内存并映射到用户页表
14         if (growproc(n) < 0) {
15             return (void*)-1;
16         }
17     }
18
19     p->heap_top += n;
20     return (void*)old_heap_top; // 返回调整前的堆顶地址
21 }

```

(四) 任务 4: 用户态系统调用接口实现

1. 桩代码生成 (参考 usys.pl)

通过 Perl 脚本自动生成 usys.S 桩代码, 示例如下:

```

1  #!/usr/bin/perl
2  # 系统调用列表
3  my @syscalls = (
4      ["fork", "SYS_fork"],
5      ["exit", "SYS_exit"],
6      ["wait", "SYS_wait"],
7      ["kill", "SYS_kill"],
8      ["getpid", "SYS_getpid"],
9      ["open", "SYS_open"],
10     ["close", "SYS_close"],
11     ["read", "SYS_read"],
12     ["write", "SYS_write"],
13     ["sbrk", "SYS_sbrk"],
14 );

```

```
15
16 # 生成汇编桩代码
17 print ".section .text\n";
18 foreach my $sc (@syscalls) {
19     my ($name, $num) = @$sc;
20     print ".global $name\n";
21     print "$name:\n";
22     print "li a7, $num\n";
23     print "ecall\n";
24     print "ret\n";
25     print "\n";
26 }
```

2. 用户库函数声明 (user.h)

```
1 // 用户态系统调用接口声明
2 int fork(void);
3 void exit(int status) __attribute__((noreturn));
4 int wait(int *status);
5 int kill(int pid);
6 int getpid(void);
7 int open(const char *path, int flags);
8 int close(int fd);
9 int read(int fd, void *buf, int count);
10 int write(int fd, const void *buf, int count);
11 void *sbrk(int n);
```

(五) 任务 5: 系统调用安全性增强

1. 缓冲区溢出防护 (sys_read 优化)

```
1 int sys_read(void) {
2     long fd, count;
3     char *buf;
4     struct proc *p = myproc();
5
6     // 提取参数
7     if (get_syscall_arg(0, &fd) < 0 || get_syscall_arg(1, (long*)&buf) <
8         0 || get_syscall_arg(2, &count) < 0) {
9         return -1;
10    }
11
12    // 缓冲区大小限制 (防止超大长度导致溢出)
13    if (count > 1024*1024) { // 限制最大读取1MB
14        count = 1024*1024;
15    }
16
17    // 其他参数检查...
18    return fileread(p->ofile[fd], buf, count);
19 }
```

```
18 }
```

2. TOCTTOU 攻击防护（文件操作原子性）

```
1  int sys_open(void) {
2      char path[PATH_MAX];
3      long flags;
4      struct inode *ip;
5      struct proc *p = myproc();
6
7      // 提取并复制路径参数（一次性复制，避免后续指针失效）
8      if (get_syscall_arg(1, &flags) < 0 || get_user_string((const char
9          __user*)p->trapframe->a0, path, PATH_MAX) < 0) {
10         return -1;
11     }
12     // 加锁保证路径解析和文件打开的原子性
13     begin_op();
14     ip = namei(path);
15     if (ip == NULL) {
16         end_op();
17         return -1;
18     }
19     // 文件打开逻辑...
20     end_op();
21
22     return fd;
23 }
```

（六）任务 6：测试与调试实现

1. 系统调用跟踪调试

```
1  void syscall_dispatch(void) {
2      struct proc *p = myproc();
3      int num = p->trapframe->a7;
4      int ret = -1;
5
6      // 调试信息输出：跟踪进程的系统调用行为
7      printf("PID %d: syscall %d (%s)\n", p->pid, num,
8          (num < NSYSCALLS) ? syscall_table[num].name : "unknown");
9
10     // 原有分发逻辑...
11 }
```

2. 基础功能测试函数

```
1  void test_basic_syscalls(void) {
2      printf("Testing basic system calls...\n");
3  }
```



```
4 // 测试getpid
5 int pid = getpid();
6 printf("Current PID: %d\n", pid);
7
8 // 测试fork与wait
9 int child_pid = fork();
10 if (child_pid == 0) {
11     printf("Child process: PID=%d, Parent PID=%d\n", getpid(), pid);
12     exit(42); // 子进程退出, 状态码42
13 } else if (child_pid > 0) {
14     int status;
15     wait(&status);
16     printf("Child (PID=%d) exited with status: %d\n", child_pid,
17           status);
18 } else {
19     printf("Fork failed!\n");
20 }
21
22 // 测试write
23 char msg[] = "Hello, Syscall!\n";
24 int bytes = write(1, msg, strlen(msg));
25 printf("Wrote %d bytes via sys_write\n", bytes);
26 }
```

四、实验关键问题解答

(一) 系统调用核心机制问题

1. 陷阱帧 (trapframe) 的作用: 存储特权级切换时的寄存器状态 (包括通用寄存器、pc、特权级状态等), 实现用户态与内核态之间的上下文保存与恢复, 确保系统调用执行完成后能回到用户程序断点继续执行。
2. 系统调用与中断处理的异同:
 - 相同点: 均会触发特权级切换, 需要保存/恢复上下文, 执行内核处理逻辑。
 - 不同点: 系统调用由用户程序主动执行 `ecall` 指令触发, 用于请求内核服务; 中断由硬件设备 (如时钟、磁盘) 触发, 异步发生, 用于处理设备事件。
3. 无法直接访问用户内存的原因: 用户态内存地址经过虚拟地址映射, 内核需通过用户页表进行地址转换; 同时用户可能传递恶意指针 (如指向内核空间的地址), 直接访问会导致内核安全漏洞。

(二) 系统调用分发与参数传递问题

1. 系统调用号传递方式: 用户态通过将系统调用号写入 `a7` 寄存器, 陷入内核后, 内核从当前进程的 `trapframe->a7` 中读取该编号。
2. 返回值存储位置: 系统调用的返回值通过 `a0` 寄存器传递, 内核将返回值写入 `trapframe->a0`, 返回用户态后, 用户程序从 `a0` 寄存器获取结果。

3. 可变参数系统调用支持方案：通过寄存器传递前 6 个参数，超过 6 个的参数存入用户栈，内核通过 `copyin()` 从用户栈读取剩余参数；或设计参数结构体，用户传递结构体指针，内核解析结构体获取所有参数。

（三）性能与安全问题

1. 系统调用主要开销：特权级切换（用户态↔内核态）、上下文保存与恢复、用户内存访问（`copyin/copyout`）、安全检查（指针验证、权限判断）。
2. 减少切换开销的优化方案：合并频繁的系统调用（如批量 IO 操作）、采用共享内存减少数据拷贝、优化上下文切换逻辑（减少寄存器保存数量）、使用快系统调用机制（如 Linux 的 `vsyscall`）。
3. 防止系统调用滥用的措施：实现权限检查（如文件访问需验证进程是否有读/写权限）、资源限制（如限制单个进程最大打开文件数）、参数严格验证（无效指针、越界长度直接拒绝）、系统调用频率限制（防止 DOS 攻击）。

（四）设计权衡与扩展性问题

1. 系统调用数量确定原则：覆盖核心功能需求（进程、文件、内存、IO 等），避免冗余调用；同时控制数量，减少内核维护成本和切换开销。
2. 功能性与安全性平衡策略：基础安全检查（指针、权限、边界）默认启用，确保底线安全；针对高性能场景，提供可选的安全降级接口（需用户显式声明并承担风险）；避免过度检查导致性能损耗。
3. 添加新系统调用的流程：①在系统调用表中添加新条目（编号、实现函数、参数个数）；②实现内核态系统调用函数；③生成对应的用户态桩代码和库函数声明；④添加参数验证、权限检查逻辑；⑤编写测试用例验证功能正确性。

（五）错误处理问题

1. 系统调用失败处理方式：返回负数值表示调用失败（如 -1 对应通用错误），同时通过 `errno` 机制存储详细错误码（需在进程结构体中添加 `errno` 字段）；用户程序通过检查返回值是否为负，再读取 `errno` 获取具体错误原因。
2. 详细错误信息报告方案：设计错误码表，每个错误码对应唯一错误描述；提供 `sys_errno()` 系统调用，用户程序可通过该调用获取当前进程的错误码及对应描述字符串。